

Object 활용실습(모델제작)

1. 프로젝트 소개



YOLO 모델을 활용해 영상 속 음식 객체를 인식하고, 프레임 단위 추론 정보를 기반으로 음식별 이미지 데이터셋을 구축하는 것을 목표로 합니다. 영상에서 특정 음식이 등장하는 시점의 프레임을 추출하여 자동 저장하고, 파일명에는 프레임 번호와 실제 시간 정보를 함께 기록합니다. 이를 통해 클래스별 데이터 불균형을 파악하고, 후속 분석(칼로리 추정, 식사 패턴 분석 등)에 활용할 수 있는 기초 데이터셋을 준비합니다.

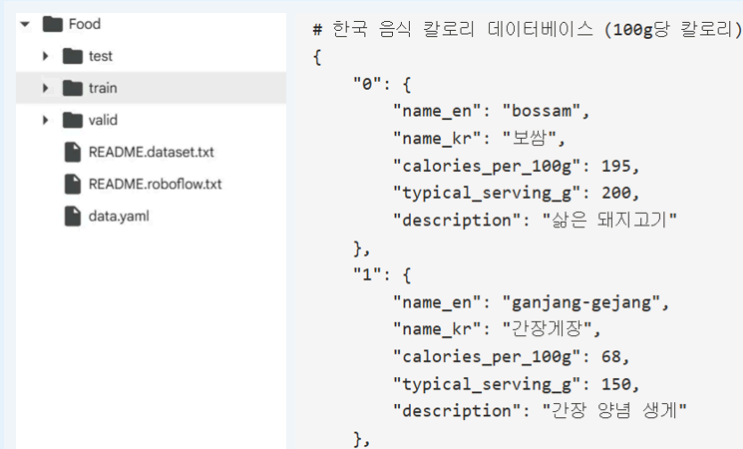
- YOLO 추론 결과를 활용하여 프레임 단위 음식 객체 탐지를 수행합니다.
- 음식별 등장 프레임을 추출하고, 폴더구조에 따라 자동 저장된 이미지 데이터셋을 구축합니다.
- 프레임 번호를 실제 시·분·초 단위로 변환하여 파일명에 반영함으로써 분석 활용성을 높입니다.
- 구축된 데이터셋을 기반으로 클래스별 분포를 확인하고, 데이터 보강 필요성을 검토합니다.
- 추후 칼로리 추정, 식사 패턴 분석, 리포트 생성 등 다양한 후속 응용 가능성을 탐색합니다.

1. Korean Food 객체인식 모델 준비 및 클래스별 칼로리 매핑

진행 프로세스

- 로보플로우에서 제공하는 한국 음식 11종 객체 인식 모델을 다운로드 받아
/content/Food 폴더에 압축 해제하여 모델 튜닝을 위한 기반을 마련합니다. 이를 통해 사전에 학습된 클래스 구조를 활용할 수 있습니다.
 - 제공된 roboflow.zip 을 다운로드 및 해제하여 모델 학습준비
!curl -L "https://app.roboflow.com/ds/MELku4OVQ7?key=b3t9dPyPwo" > roboflow.zip
- 객체 인식 결과를 단순 검출에 그치지 않고, 칼로리 산출 및 영양 분석까지 확장할 수 있도록 data.yaml 파일을 통해 11개 음식 클래스를 확인한 뒤, 각 음식에 대응하는 칼로리 정보를 별도 JSON 파일로 구성합니다.
 - 클래스별 칼로리 정보를 /content/food_calories.json 으로 저장
(제시된 JSON 출력은 예시이며, 포함할 정보와 구조는 자유롭게 정의가능)

결과물



2. Korean Food 객체 인식: YOLO 모델 학습

진행 프로세스

- 속도와 정확성에서 강점을 지닌 YOLO를 기반으로 객체 인식 모델을 학습시키며, 가장 성능이 좋은 모델을 확보해 이후 재사용과 확장을 쉽게 하기 위하여 학습과정의 최적의 결과를 저장합니다.

- 사전 학습된 YOLO 모델을 선택하여 학습 환경 구성(옵션은 자유롭게)
- 데이터셋 학습 진행 → best 모델 가중치 저장

▼ 코드참조(yolov8s.pt 모델과 에포크 50번, 추후50번하여 총 100번의 결과물 코드 참조)

50번에포크 추가한 모델이 성능이 떨어졌음. 이를 고려하여 처음의 best모델을 보존해야함

```
from ultralytics import YOLO

# 1. 사전학습된 YOLO 모델 불러오기 (nano → 빠름, small → 조금 더 정확)
model = YOLO("yolov8s.pt") # 필요하다면 "yolov8s.pt" 사용 가능

# 2. 모델 학습 실행
results = model.train(
    data="Food/data.yaml", # 데이터셋 경로
    epochs=50,             # 학습 횟수 (5 → 50으로 증가)
    imgsz=640,             # 입력 이미지 크기
    batch=16,              # Colab T4/L4 GPU에 적합
    device=0,              # GPU 사용
    patience=10,           # 조기 종료 여유를 넉넉하게
    save=True,             # 체크포인트 저장
    project="food_detection", # 결과 저장 폴더
    name="exp",             # 실험명
    exist_ok=True,         # 폴더 덮어쓰기 허용
    plots=True,            # 학습 곡선 저장
    optimizer="Adam"       # Adam 옵티마이저 (SGD보다 안정적인
    때 있음)
)

# 3. 학습된 best 모델 불러오기
best_model = YOLO("food_detection/exp/weights/best.pt")
```

4. 테스트 세트 성능 평가 (정확한 지표 확인)

```
metrics = best_model.val(data="Food/data.yaml", split="test")
print("mAP50:", metrics.box.map50)
print("mAP50-95:", metrics.box.map)
print("Precision:", metrics.box.mp)
print("Recall:", metrics.box.mr)
```

5. 테스트 이미지로 예측 (결과 저장)

```
results = best_model.predict(
    source="Food/test/images",
    save=True,          # 결과 저장
    conf=0.25,          # 낮춰서 놓치는 객체 줄이기
    project="food_detection",
    name="predictions"
)
```

```
print(" 학습 완료 & 테스트 평가 완료!")
print(" 학습 결과: food_detection/exp/")
print(" 예측 결과: food_detection/predictions/")
```

```
#####33
## 모델을 추가 학습하고자 함
#####
from ultralytics import YOLO
```

best 모델 불러오기

```
model = YOLO("food_detection/exp/weights/best.pt")
```

추가 학습 진행 (예: 50 epoch 더)

```
results = model.train(
    data="Food/data.yaml",
    epochs=50,          # 새로 50 epoch 학습
    imgsz=640,
    batch=16,
    device=0,
    patience=10,
    save=True,
```

```

project="food_detection",
name="exp_best_continue", # 새로운 폴더에 저장
exist_ok=True
)

```

- 모델이 실제 상황에서 얼마나 잘 동작하는지 검증하고 신뢰할 수 있는 성능을 확보하기 위하여 'best.pt' 파라미터로 test의 이미지를 시각화합니다.
 - **POC(초기 확인):** 전체 **mAP@0.5 $\geq$ 0.70**, **핵심 클래스는 $\geq$ 0.80**, **최소 클래스별 $\geq$ 0.50**
 - 테스트 이미지 예측 → 결과 저장 및 검토

결과물

결과물은 예시이며, 학습 과정(데이터 양·품질, 에폭 수 등)에 따라 결과는 달라집니다.

2.(참고) 모델 훈련시 생성되는 시각화를 통하여서 품질 수준 확인가능

- 처음 50번의 에포크

Model summary (fused): 72 layers, 11,129,841 parameters, 0 gradients, 28.5 GFLOPs

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%
all	526	858	0.754	0.752	0.761	0.673
bossam	3	6	0	0	0	0
ganjang-gejang	22	22	0.931	0.955	0.954	0.901
gimbap	41	184	0.958	0.978	0.992	0.877
kimchi	153	192	0.79	0.885	0.862	0.554
kkakdugi	41	43	0.969	0.716	0.912	0.767
ramyun	56	56	0.917	0.981	0.973	0.854
sundae gukbap	254	254	0.993	0.996	0.995	0.96
tteokbokki	41	45	0.248	0.267	0.168	0.154
yangpa jangajji	56	56	0.982	0.988	0.995	0.991

Speed: 0.1ms preprocess, 1.3ms inference, 0.0ms loss, 0.9ms postprocess per image
Results saved to /content/food_detection/exp_best_continue

- best.pt모델을 불러와 50번의 에포크 추가후

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%
all	258	423	0.826	0.82	0.86	0.743
bossam	1	2	0	0	0	0
ganjang-gejang	13	13	0.779	1	0.974	0.864
gimbap	25	112	0.969	0.92	0.982	0.855
kimchi	79	100	0.794	0.86	0.853	0.552
kkakdugi	11	11	0.884	0.696	0.957	0.763
ramyun	25	25	0.96	0.956	0.99	0.851
sundae	7	7	0.937	0.857	0.866	0.714
sundae gukbap	109	109	0.984	1	0.995	0.948
tteokbokki	22	24	0.994	0.917	0.984	0.897
yangpa jangajji	20	20	0.96	1	0.995	0.987

Speed: 0.6ms preprocess, 2.6ms inference, 0.0ms loss, 2.6ms postprocess per image

3. 모델 훈련시 생성되는 시각화

본 시각화 분석을 통해 모델의 성능을 평가하고, 추가 데이터 보강이 필요한 클래스를 확인합니다.

- 모든 작업을 완료한 후 다시 검토하며, 데이터셋 품질이 성능에 큰 영향을 미치므로 필요 시 데이터셋 변경도 고려해야 합니다.

결과물 (에포크5번, yolov8n.pt 경량화 모델 결과임. 2번의 모델로 실행했을때의 결과와 비교)

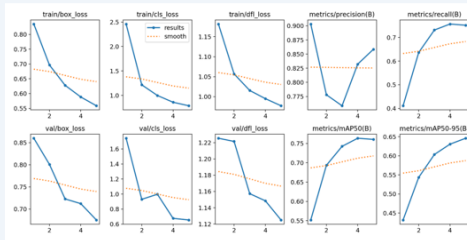
Sample Predictions (예측 결과 시각화 이미지)

predict/ 폴더에 저장된 실제 이미지에서 박스가 잘 그려졌는지, 누락이나 과검출이 있는지 육안 확인 모델 숫자 지표보다 현업에서는 가장 많이 보는 부분



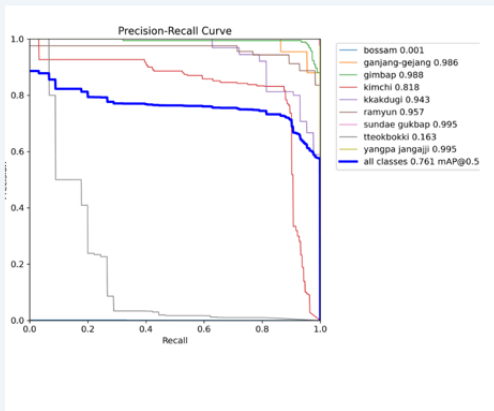
result.png

Training loss / Validation loss → 과적합 여부 확인
mAP, Precision, Recall 곡선 → 학습이 잘 진행되었는지 추세 확인



BoxPR_curve.png

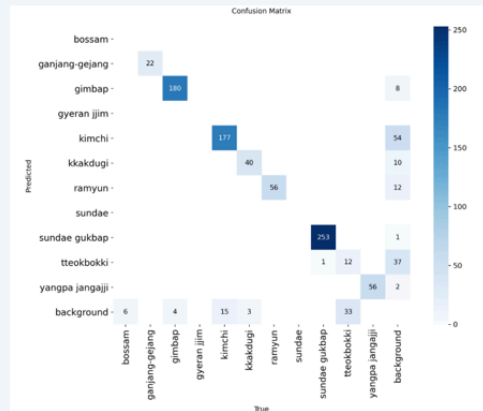
각 클래스별 성능 균형을 보여줌.
특정 클래스는 Precision은 높지만 Recall이 낮을 수 있는데, 이런 경우 데이터 증강/보강 필요성 확인 가능



confusion_matrix.png

어떤 클래스를 잘못 분류했는지 한눈에 볼 수 있음.

예: 김밥과 김치가 서로 혼동되는 경우
→ 데이터셋 라벨링/샘플 개선 포인트

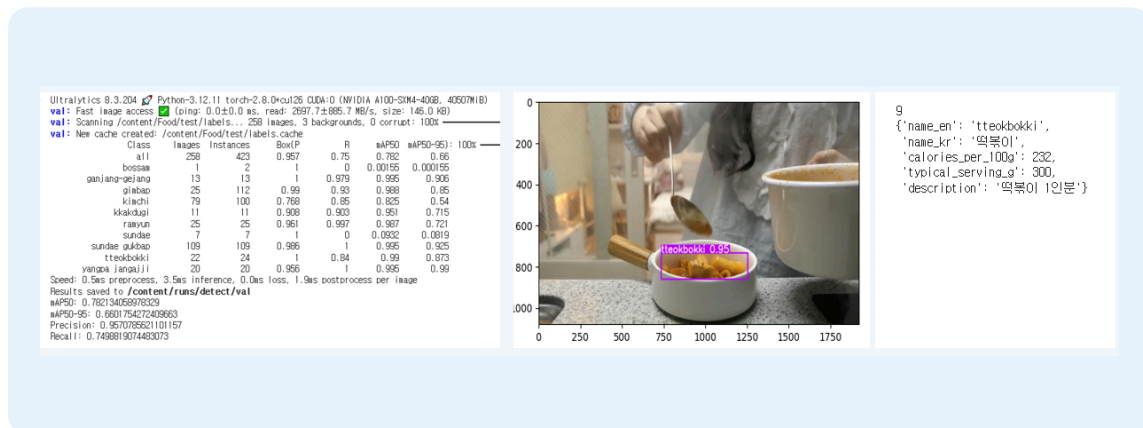


4. YOLO 추론 결과와 영양 정보 매핑

진행 프로세스

- **테스트 세트 성능 평가:** 학습된 `best.pt` 모델을 불러와 테스트 세트로 성능을 검증합니다.
 - mAP50, mAP50-95, Precision, Recall 값을 출력하여 모델의 전반적인 성능 확인
- 테스트 이미지 한 장을 입력하여 **이미지 추론 실행을 실행하며, 칼로리 정보를 연동하여** 모델이 예측한 클래스 ID를 이용해 해당 음식의 칼로리 정보를 조회하고 출력합니다.
 - 예측된 결과는 시각화하여 확인하고, 탐지된 음식 객체의 위치와 클래스 정보 출력
 - `food_calories.json` 파일에서 음식 클래스에 해당하는 칼로리 값 출력

결과물



5. 동영상 추론 & 영양정보 매핑

진행 프로세스

- 다음 코드를 이용하여 유튜브 영상자료를 다운로드합니다.

▼ 유튜브 영상자료 다운로드 코드

유튜브 url은 예이며 직접 모델 제작 source데이터셋 확인하여 선택하여 봅니다

```
# YouTube 영상다운로드
# 1. 필요한 패키지 설치
!pip install yt-dlp

# 2. YouTube 영상 다운로드
video_url="https://www.youtube.com/watch?v=tCxH2zS5tCw"

# 영상 다운로드 (전체 또는 일부)
!yt-dlp -f "best[height<=720]" -o "food.mp4" {video_url}
# 처음 30초만 다운로드하려면:
# !yt-dlp -f "best[height<=720]" --download-sections "*0-30"
-o "food_video.mp4" {video_url}

print("영상 다운로드 완료")
```

- 영상의 일부분만 테스트 한후 (3초정도), 동영상의 추론정보, 칼로리정보를 매핑한 데이터를 작성한뒤 영상자료에 대한 기초통계와 시각화로 추론결과를 분석합니다.
 - save_txt=True 옵션을 통하여 각 프레임별로의 영상 추출 기록을 저장

▼ 코드 참조

```
import cv2
from ultralytics import YOLO

# 1. YOLO 모델 불러오기
model = YOLO("food_detection/exp/weights/best.pt")

# 2. 원본 영상 열기
video_path = "/content/food.mp4"
cap = cv2.VideoCapture(video_path)

fps = int(cap.get(cv2.CAP_PROP_FPS))
```

```

width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

# 3. 앞 3분만 잘라서 새 영상 저장
clip_path = "/content/food_clip_3min.mp4"
out = cv2.VideoWriter(clip_path, cv2.VideoWriter_fourcc(
    *'mp4v'), fps, (width, height))

max_frames = fps * 180 # 3분(180초)
frame_count = 0

while cap.isOpened():
    ret, frame = cap.read()
    if not ret or frame_count >= max_frames:
        break
    out.write(frame)
    frame_count += 1

cap.release()
out.release()
print(f"🎬 앞 3분(총 {frame_count}프레임) 저장 완료: {clip_path}")

# 4. YOLO 추론 (앞 3분 영상만)
results = model.predict(
    source=clip_path,
    conf=0.5,
    save=True, # 결과 영상 저장
    show=False,
    stream=False,
    save_txt=True
)

print(f"\n✅ 총 {frame_count}프레임(3분) 분석 완료!")
print(f"📁 결과 영상은 runs/detect/predict/ 에 저장됩니다.")

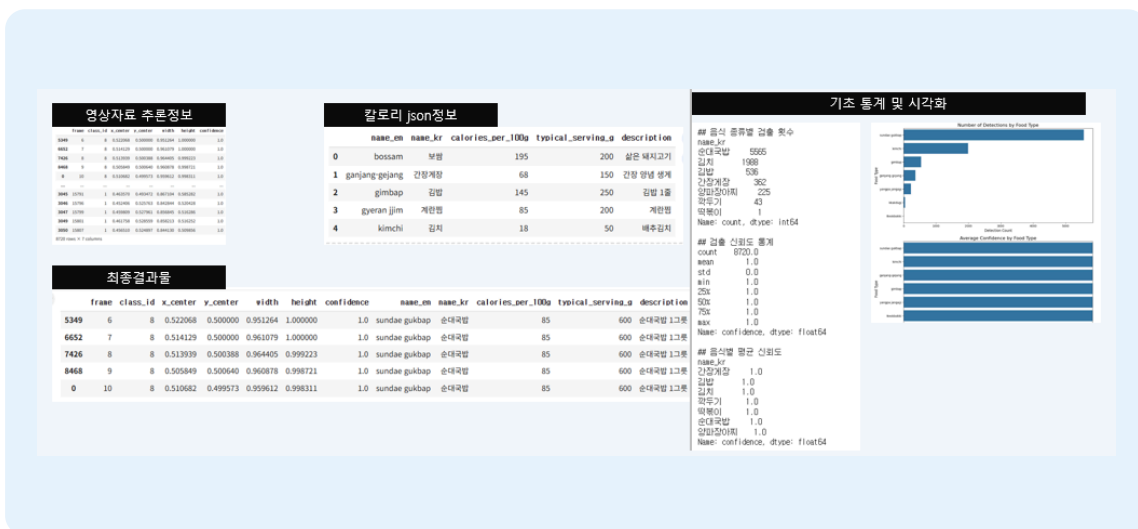
```

- 추론결과의 label폴더에 있는 결과물.txt의 모든 취합, 기초통계

- 동영상 데이터는 초당 수십 장의 프레임으로 구성되며, 객체 인식 모델은 이를 개별 이미지로 분석합니다. 그러나 모든 프레임을 직접 분석하면 데이터가 과도하게 세분화되어 전체적인 흐름을 파악하기 어렵습니다. 따라서 1초단위 이동 평균을 적용해 변동성을 줄이고, 음식 등장 패턴을 보다 명확하게 시각화합니다.
 - 매 프레임마다 검출된 음식 객체의 개수를 라인차트로 시각화
 - food.mp4의 FPS로 계산한 음식 객체 수의 이동 평균을 라인차트로 시각화

결과물

- 추론 라벨링 취합 및 기초통계 시각화

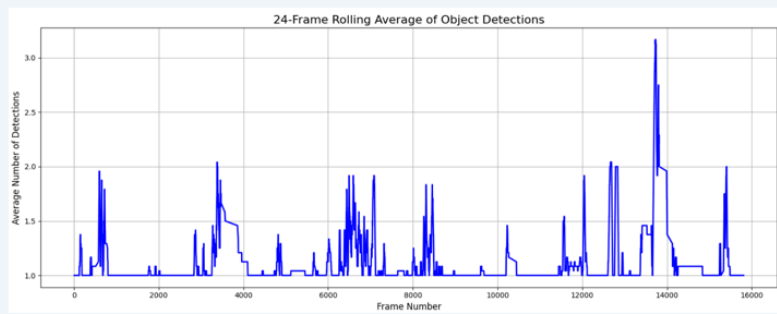
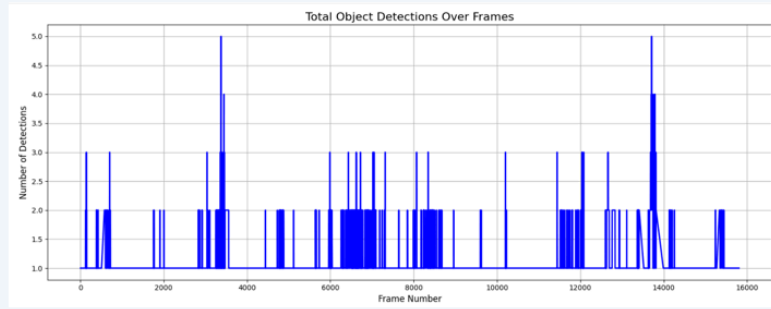


- 음식 등장 패턴 시각화

매프레임마다의
음식객체수



동영상 프레임
단위의 평균선



6. 프레임 단위 추론 결과를 활용한 클래스별 이미지 정리

진행 프로세스

- 음식 이름별로 이미지를 정리해두면 객체 인식·분류 모델의 학습용 데이터셋으로 직접 활용할 수 있으며, 클래스별 데이터 분포를 비교해 불균형을 확인하고 추가 수집이나 보강이 필요한 대상을 쉽게 파악할 수 있습니다. 또한 이렇게 저장된 프레임 이미지는 칼로리 추정, 식사 패턴 분석, 리포트 생성 등 다양한 후속 분석과 응용으로 확장할 수 있습니다.
 - 데이터를 활용해 음식 이름별로 등장하는 프레임을 탐색하고, 해당 프레임 이미지를 추출하여 음식 이름 폴더에 자동 저장
 - 프레임 번호를 실제 시·분·초로 변환하여 음식별 폴더에 저장된 이미지 파일명을 일관된 형식으로 정리
- 최종 출력물을 이용한 활용방안 아이디어를 도출합니다.

결과물

클래스별 영상 위치(예시)

```
1 frames_by_food = df_merged.groupby('name_kr')['frame'].unique()
2 frames_by_food
```

name_kr	frame
간장게장	[51, 144, 145, 148, 194, 279, 280, 281, 282, 2...
김밥	[255, 519, 544, 549, 551, 554, 555, 556, 557, ...
김치	[38, 39, 40, 41, 42, 43, 44, 45, 235, 236, 237...
깍두기	[14544, 14545, 14546, 14547, 14548, 14554, 152...
떡볶이	[1448, 1690, 1691, 1692, 1693, 1694, 14510, 14...
라면	[9301, 9306, 9307]
순대국밥	[6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 1...
양파장아찌	[4573, 4574, 4575, 4659, 4660, 4661, 4662, 466...

출력예시

```
/content/food_frames_img/
├─ gimbap/
│   ├── gimbap_255frame_00-00-10.jpg
│   ├── gimbap_519frame_00-00-21.jpg
│   └── gimbap_544frame_00-00-22.jpg
├─ ramyun/
│   ├── ramyun_9301frame_06-27-05.jpg
│   └── ramyun_9306frame_06-27-06.jpg
└─ tteokbokki/
    ├── tteokbokki_1448frame_01-00-20.jpg
    └── tteokbokki_1690frame_01-10-25.jpg
```